

과제보고서 제출서약서

나는 승실대학교 정보보호학과의 일원으로 명예를 지키면서 생활하고 있습니다.

나는 보고서를 작성하면서 다음과 같은 사항을 준수하였음을 엄숙히 서약합니다.

1. 나는 자력으로 보고서를 작성하였습니다.
2. 나는 보고서에서 참조한 문헌의 출처를 밝혔으며 표절하지 않았습니다.
3. 나는 보고서의 내용을 조작하거나 날조하지 않았습니다.

교 과 목	자료구조
과 제 명	정렬 기법 성능 비교
교과목 교수	이상준 교수
제 출 인	정보보호학과 2 학년 학번: 20245027 성명: 유창하
제 출 일	2025년 5 월 16 일 (금)

1. 과제 소개

비교 기반 정렬 알고리즘 6가지를 직접 구현해보고, 서로의 성능을 비교하는 프로그램입니다. 데이터 크기를 100, 1,000, 10,000, 100,000으로 나누고, 동일한 난수 데이터를 여러 개의 파일로 저장한 뒤, 정렬 알고리즘마다 다른 파일을 불러오도록 설계하여 캐시 효과를 방지하였습니다. 각 알고리즘의 정렬 시간(clock() 함수 사용)과 비교 횟수를 측정하여 결과를 출력했습니다.

2. 프로그램 구조

sorting_project/

```
|—— data/                                ← 입력 데이터 파일 폴더
|   |—— data_100.txt
|   |—— data_100_copy1.txt
|   |—— ...
|—— generate_data.c                       ← 랜덤 숫자 데이터 생성 및 복사본 만들기
|—— sort.c                               ← 6가지 정렬 알고리즘 구현 및 성능 측정
|—— Makefile                             ← 컴파일 자동화 도구
```

1. generate_data.c

데이터 파일을 생성하는 프로그램입니다. 하나의 랜덤 데이터 배열을 만든 뒤, 같은 내용을 여러 개의 복사본 파일로 저장합니다. 캐시의 영향을 줄이기 위해, 이 방법을 사용하였습니다.

2. sort.c

실제 정렬 알고리즘을 구현하고, 성능을 측정하는 메인 프로그램입니다. Bubble, Selection, Insertion, Shell, Merge, Quick 정렬 구현을 구현하였고, clock()을 이용한 실행 시간 측정하였습니다. 비교 연산 횟수를 세기 위한 Metrics 구조체를 따로 만들어, 비교연산을 할 때마다 횟수가 증가됩니다.

3. Makefile

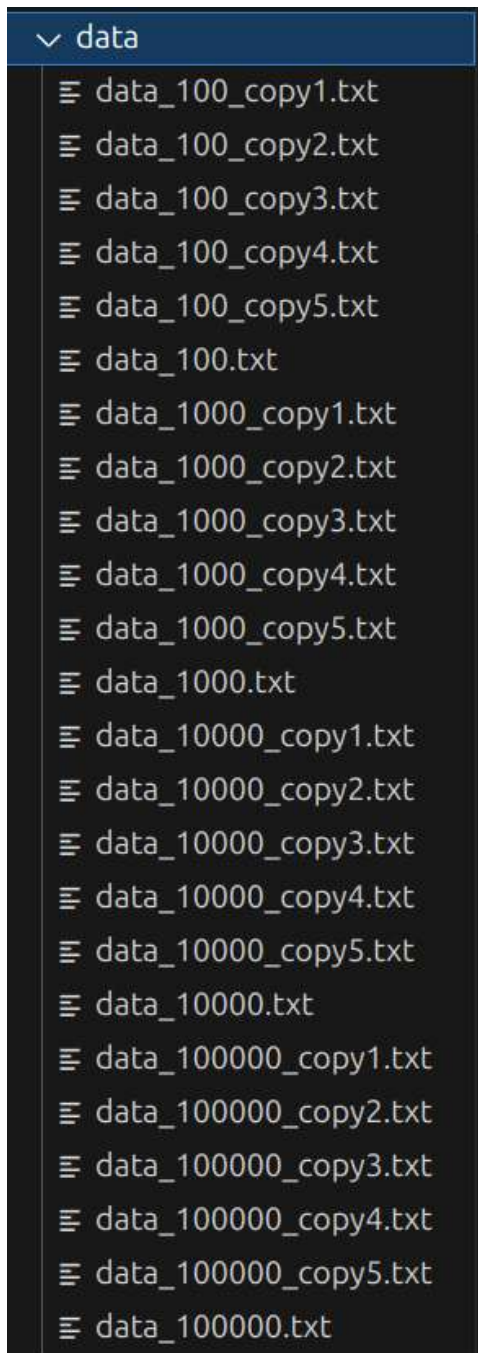
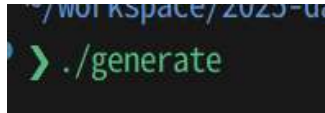
터미널에서 make 명령어로 프로그램을 쉽게 컴파일할 수 있도록 도와주는 파일입니다.

```
$ make          # generate와 sort 프로그램 모두 컴파일
$ ./generate    # 데이터 파일 생성
$ ./sort        # 정렬 알고리즘 성능 비교 실행
```

3. 과제 소스 코드

<https://github.com/yc54616/2025-data-struct/tree/problem3>

4. 결과 실행 화면



```
> ./sort

=== 데이터 크기: data/data_100.txt ===
Bubble      | 비교 횟수:   4950 | 시간: 0.0000초
Selection    | 비교 횟수:   4950 | 시간: 0.0000초
Insertion    | 비교 횟수:  2513 | 시간: 0.0000초
Shell        | 비교 횟수:   870 | 시간: 0.0000초
Merge        | 비교 횟수:   545 | 시간: 0.0000초
Quick        | 비교 횟수:   748 | 시간: 0.0000초

=== 데이터 크기: data/data_1000.txt ===
Bubble      | 비교 횟수:  499500 | 시간: 0.0018초
Selection    | 비교 횟수:  499500 | 시간: 0.0010초
Insertion    | 비교 횟수:  249389 | 시간: 0.0003초
Shell        | 비교 횟수:  15694 | 시간: 0.0001초
Merge        | 비교 횟수:   8720 | 시간: 0.0001초
Quick        | 비교 횟수:  10300 | 시간: 0.0001초

=== 데이터 크기: data/data_10000.txt ===
Bubble      | 비교 횟수: 49995000 | 시간: 0.1372초
Selection    | 비교 횟수: 49995000 | 시간: 0.0547초
Insertion    | 비교 횟수: 25224690 | 시간: 0.0395초
Shell        | 비교 횟수:  259229 | 시간: 0.0014초
Merge        | 비교 횟수:  120450 | 시간: 0.0012초
Quick        | 비교 횟수:  150673 | 시간: 0.0009초

=== 데이터 크기: data/data_100000.txt ===
Bubble      | 비교 횟수: 4999950000 | 시간: 21.8700초
Selection    | 비교 횟수: 4999950000 | 시간: 5.3784초
Insertion    | 비교 횟수: 2507242813 | 시간: 3.9997초
Shell        | 비교 횟수:  4268764 | 시간: 0.0203초
Merge        | 비교 횟수:  1536552 | 시간: 0.0144초
Quick        | 비교 횟수:  2070474 | 시간: 0.0103초
```

5. 결론

이번 과제를 통해 각 알고리즘이 어떤 한계를 해결하기 위해 고안되었는지, 그리고 그 의도가 실제 성능에서 어떻게 나타나는지를 확인할 수 있었습니다.

먼저, 버블 정렬은 구현이 매우 단순하지만, 비교와 swap 연산이 과도하게 많아 성능이 매우 떨어졌습니다. 특히 데이터 크기가 커질수록 실행 시간이 크게 증가했습니다.

이러한 버블 정렬의 단점을 보완하기 위해 등장한 선택 정렬은 비교 횟수는 여전히 많지만, swap 연산을 한 번만 수행하여 불필요한 이동을 줄인다는 특징이 있습니다. 버블 정렬보다 약간 더 나은 성능을 보였지만, 비교 횟수의 한계로 인해 전체적인 성능 차이는 크지 않았습니다.

다음으로, 삽입 정렬은 기존에 정렬된 데이터를 유지하며 새로운 데이터를 알맞은 위치에 삽입하는 방식으로, 불필요한 비교와 이동을 줄인 것을 눈으로 확인할 수 있었습니다. 특히 데이터가 어느 정도 정렬된 상태일수록 빠르게 작동하는 특성이 잘 나타났습니다.

하지만 삽입 정렬은 여전히 많은 shift 연산을 포함하고 있어 데이터 양이 많아질 경우 성능 저하가 발생합니다. 이를 해결하기 위한 셸 정렬은, 간격을 두고 삽입 정렬을 반복함으로써 전체 이동 횟수를 줄이는 효과를 가져왔습니다. 비교 횟수와 실행 시간이 모두 눈에 띄게 줄어들었으며, 삽입 정렬의 확실한 개선 버전임을 확인할 수 있었습니다.

마지막으로, 병합 정렬과 퀵 정렬은 분할 정복 기법을 활용하는 고급 정렬 방식으로, 시간 복잡도 $O(n \log n)$ 을 이론적으로 가장 빨랐고, 실제 정렬에서도 가장 뛰어난 성능을 보였습니다.